

UNIVERSIDAD DE LAS AMERICAS

PROYECTO MIA – MAESTRIA INTELIGENCIA ARTIFICIAL APLICADA

S10 — Avance Consolidado del Proyecto y Documento Final



Grupo: Patricio Bayas Meza, José Puebla Paladines, Marco Zurita Rojas

Fecha: 23 de agosto de 2026

Resumen

Las historias clínicas contienen inconsistencias medicación contraindicada, incoherencias entre diagnóstico y tratamiento, contradicciones internas— que comprometen la seguridad del paciente.

Este informe consolida el avance del proyecto MIA_Deteccion_HC para CITIMED: asistente con TF-IDF, LLM zero-shot y LLM+RAG evaluados sobre MEDEC. El baseline léxico a nivel de nota no supera el azar (ROC-AUC 0,504).

Al reformular la detección a nivel de oración, el modelo ajustado alcanza ROC-AUC 0,949 (IC 95 % 0,940–0,958), AUPRC 0,419 (prevalencia 4,5 %) y localiza la oración errónea en 84,6 % (263/311 notas). Se entregaron comparación tripartita, índice FAISS+RAG, demo Streamlit, API FastAPI, frontend React, fallback TF-IDF, evaluaciones por tipo de error y sesgo EN-ES. Como avance hacia CITIMED, se implementó anonimización local (s10/anonimizador/) y eval cross-domain sobre plantilla odontológica etiquetada (n=4): ROC-AUC 1,000, AUPRC 1,000; inferencia piloto sobre 1 historia(s) anonimizada(s) (10 oraciones, sin mezclar en métricas AUC).

Corpus clínico anotado y aval ético pendientes.

Repositorio: https://github.com/marcoz1691/MIA_Deteccion_HC.

Palabras clave: historias clínicas, detección de inconsistencias, LLM, RAG, MEDEC, seguridad del paciente, MLOps.

1. Introducción

1.1. Problema y contexto organizacional

La Clínica CITIMED gestiona historias clínicas electrónicas que combinan texto libre (notas de evolución, epicrisis) y campos estructurados (diagnósticos CIE-10, medicación, laboratorio). Los errores de documentación son frecuentes y clínicamente relevantes: la literatura reporta que uno de cada cinco pacientes que leen su nota detecta un error. Estas inconsistencias pueden derivar en decisiones equivocadas y afectar la seguridad del paciente. La revisión manual exhaustiva no es viable por el volumen de registros.

1.2. Objetivo

Diseñar e implementar un asistente inteligente que, dada una historia clínica, detecte inconsistencias, localice la oración afectada y sugiera una corrección apoyada en conocimiento médico externo (guías, CIE-10, vademécum, protocolos institucionales), sirviendo de apoyo — no de reemplazo— al criterio clínico.

1.3. Estado del arte (síntesis)

La literatura reciente (2023-2025) muestra tres tendencias convergentes. Primera: los LLM han pasado de responder preguntas médicas a validar la corrección y consistencia del texto clínico; MEDEC (Ben Abacha et al., 2025) es el primer banco público anotado para detección y corrección de errores en notas. Segunda: la inconsistencia y la alucinación son el principal obstáculo para el uso clínico, y un error en la entrada tiende a propagarse. Tercera: la generación aumentada por recuperación (RAG) es la estrategia dominante de mitigación, pues anclar las respuestas en fuentes externas reduce las alucinaciones y aporta trazabilidad. En conjunto, el

estado del arte respalda la arquitectura elegida y advierte sobre privacidad, evaluación estandarizada y sesgos (incluido el sesgo en español), aspectos que este proyecto incorpora explícitamente.

2. Metodología

2.1. Datos y supuestos

Para el desarrollo y la evaluación reproducible se usa el banco público MEDEC, con sus particiones oficiales (2.189 notas de entrenamiento, 574 de validación y 597 de prueba con verdad de referencia) y cinco tipos de error: manejo, diagnóstico, farmacoterapia, tratamiento y organismo causal. Se eligió MEDEC porque los datos reales de CITIMED están en proceso de anonimización y aval ético. Supuesto principal: cada nota contiene a lo sumo una inconsistencia localizable a nivel de oración. Como el banco está en inglés y en contexto estadounidense, se asume una brecha de dominio e idioma respecto de CITIMED, que se auditará en la fase final.

2.2. Herramientas y enfoque (low-code / código abierto)

El pipeline se implementó con herramientas de código abierto: scikit-learn (TF-IDF + Regresión Logística), cliente OpenAI-compatible para LLM (s7/llm_client.py), sentence-transformers + FAISS para RAG (s7/rag_index.py, s7/knowledge/), inferencia unificada (s7/inferencia.py), demo Streamlit (demo/), API REST FastAPI (api/) y frontend React (frontend/). Para CITIMED se añadió el servicio de anonimización (s10/anonimizador/ANONIMIZADOR/: reglas Ecuador + spaCy NER), utilidades compartidas s10/citimed_utils.py y s10/probar_citimed.py (anonimizar → inferencia ES → eval cross-domain sobre plantilla etiquetada). Mock LLM activo por defecto; soporte Ollama on-premise (OPENAI_BASE_URL=http://localhost:11434/v1) para PHI.

Trabajo futuro: Orquestación LangChain/LlamaIndex, Docker Compose, DVC, MLflow operativo (requirements-optional.txt, entorno separado). Reproducibilidad: SEED=42, particiones oficiales MEDEC, métricas en salidas_ajuste/ y salidas_s7/.

2.3. Enfoque de modelado seguido

- **Baseline (punto de comparación).** TF-IDF (n-gramas 1-2) + Regresión Logística para clasificar la nota como “con/sin inconsistencia”. Sencillo, determinista e interpretable.
- **Modelo ajustado.** Reformulación a nivel de oración (“¿es esta oración la inconsistente?”), con ingeniería de características (TF-IDF + rasgos numéricos por oración mediante FeatureUnion) y ajuste de hiperparámetros por validación cruzada estratificada de 5 pliegues.

- **Evaluación.** Métricas de clasificación (accuracy, precision, recall, F1, ROC-AUC), una línea de referencia trivial, intervalos de confianza por bootstrap, prueba de McNemar para comparaciones pareadas y una métrica de negocio: la localización top-1 de la oración errónea.

Actualización S10:

- Comparación tripartita TF-IDF / LLM / LLM+RAG: IMPLEMENTADA (s7/eval_tripartita.py; subset n=500, mock LLM=True).

TF-IDF ROC-AUC=0,954; LLM zero-shot=0,498; LLM+RAG=0,498. McNemar TF-IDF vs LLM: $p=0,111$.

- Corpus CITIMED en español: AVANCE PILOTO S10 — anonimizador funcional (s10/anonimizador/); inferencia piloto sobre 1 historia(s) anonimizada(s) (10 oraciones, sin mezclar en métricas AUC); eval cross-domain sobre plantilla odontológica etiquetada (n=4): ROC-AUC 1,000, AUPRC 1,000. Corpus clínico anotado y aval comité de ética: PENDIENTE.
- MLflow operativo: PLAN FUTURO (requirements-optional.txt; venv separado numpy<2).

3. Resultados

3.1. Baseline sobre MEDEC (evaluación previa, S6)

El baseline a nivel de nota obtuvo, en la partición de prueba, un ROC-AUC de 0,504: desempeño equivalente al azar, sin superar al clasificador trivial (accuracy 0,523 frente a 0,521). El F1 de 0,674 resultó engañoso, pues el modelo marcaba casi todas las notas como erróneas. El diagnóstico del corpus explicó el resultado: la nota con error y su versión corregida son idénticas en un 96,6 %, la oración errónea representa apenas el 8,9 % del texto y el vocabulario se solapa un 95,6 % entre clases. Una ablación de cinco variantes confirmó que el límite es representacional, no de ajuste.

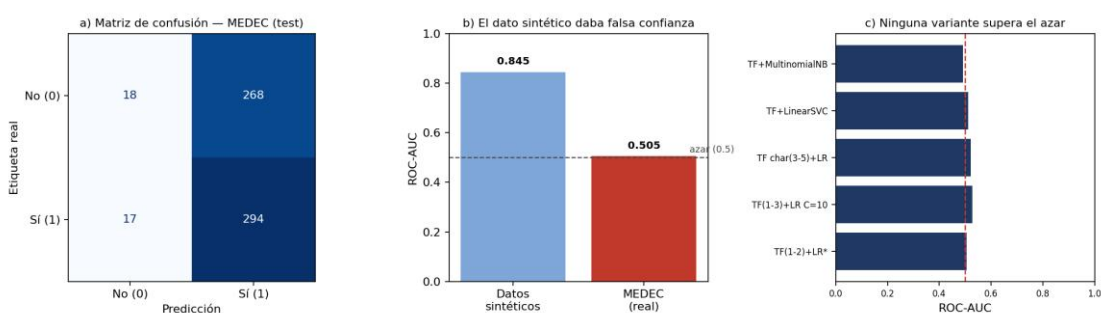


Figura 1 (S6). (a) el baseline clasifica casi todo como error; (b) los datos sintéticos daban una confianza falsa (AUC 0,845) frente al dato real (0,504); (c) ninguna variante supera el azar.

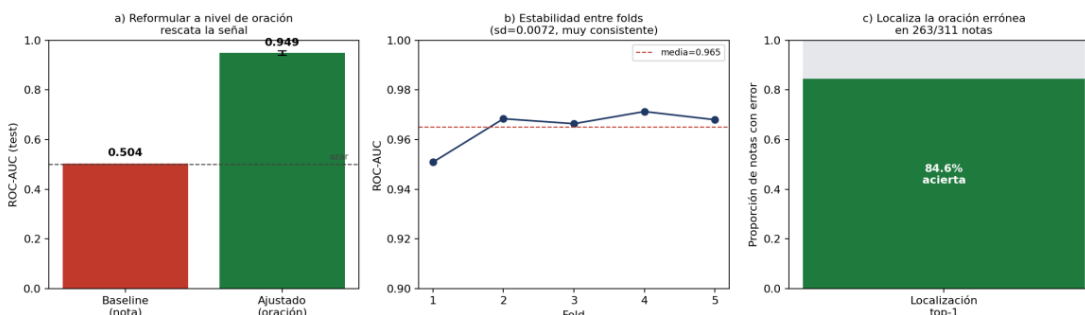


Figura 1. Curvas ROC/PR y lift — baseline vs ajustado (S6).

3.2. Modelo ajustado a nivel de oración (S7)

Reformular la tarea a nivel de oración recuperó el poder discriminante. La tabla resume la comparación sobre la misma partición de prueba de MEDEC.

Métrica (test MEDEC)	Baseline (nota)	Ajustado (oración)	Lectura
ROC-AUC	0.504	0,949	De azar a fuerte discriminación
ROC-AUC (IC 95 %)	—	0,940 – 0,958	Intervalo estrecho: resultado robusto
CV 5-fold (AUC)	—	0,965 ± 0,007	Muy estable entre pliegues
Recall (oración)	0.945*	0.817	*el del baseline era espurio
Precision (oración)	0.523	0.357	Baja por desbalance (4,5 % positivos)
Localización top-1	n/d	0,846	Acierta la oración en 263/311 notas
AUPRC (oración)	—	0,419	IC 95 % 0,392–0,446; métrica clave con 4,5 % positivos

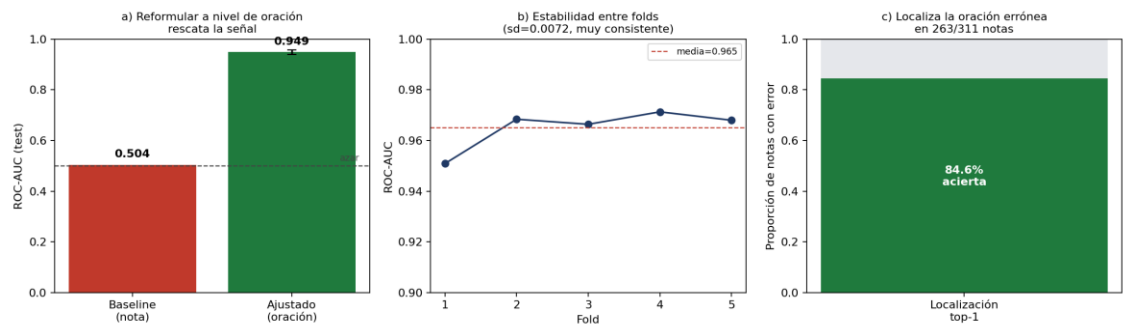


Figura 2 (S7). (a) salto de ROC-AUC al pasar a nivel de oración, con IC 95 % por bootstrap; (b) estabilidad entre folds; (c) localización top-1 de la oración errónea.

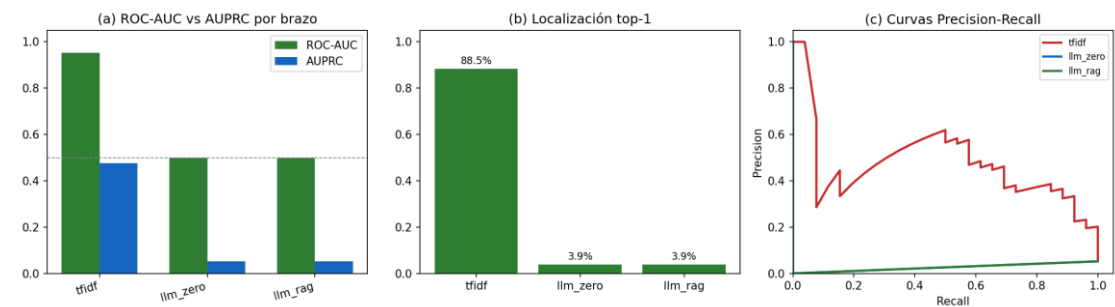


Figura 2. Comparación tripartita TF-IDF / LLM / LLM+RAG.

Un matiz importante: al reagregar las predicciones a nivel de nota, el modelo ajustado no mejora al baseline (prueba de McNemar, $p = 0,53$). Esto es esperable, porque en MEDEC casi toda nota contiene una oración candidata a error; el valor del ajuste está en la localización de la oración, que es la capacidad útil para el revisor clínico, no en la clasificación binaria de la nota.

3.3. Piloto CITIMED — anonimización e integración (avance S10)

Como avance hacia el despliegue en CITIMED (no como corpus clínico final), se implementó el servicio de anonimización en `s10/anonizador/ANONIMIZADOR/` (reglas para identificadores ecuatorianos + NER spaCy, pseudonimización consistente, fechas desplazadas y auditoría sin PHI en claro). El script `s10/probar_citimed.py` ejecuta el flujo: historia → anonimizar → inferencia en español (TF-IDF + LLM mock). La evaluación cross-domain usa solo la plantilla odontológica etiquetada; las historias anonimizadas del piloto se reportan aparte (sin mezclar labels artificiales).

Inferencia piloto: 1 historia(s) anonimizada(s), 10 oraciones segmentadas. Eval cross-domain sobre plantilla odontológica etiquetada (n=4 oraciones): ROC-AUC = 1,000, AUPRC = 1,000. Estas cifras son preliminares (subset pequeño) y no sustituyen validación con corpus CITIMED anotado y aval del comité de ética. Reporte: `salidas_s7/prueba_citimed.json`.

Limitaciones del piloto: historias de ejemplo sintéticas; métricas cross-domain sensibles al tamaño muestral; anonimización automática requiere revisión humana por muestreo antes de uso clínico. Trabajo pendiente: lote anonimizado institucional, anotación de inconsistencias y evaluación con Ollama on-premise.

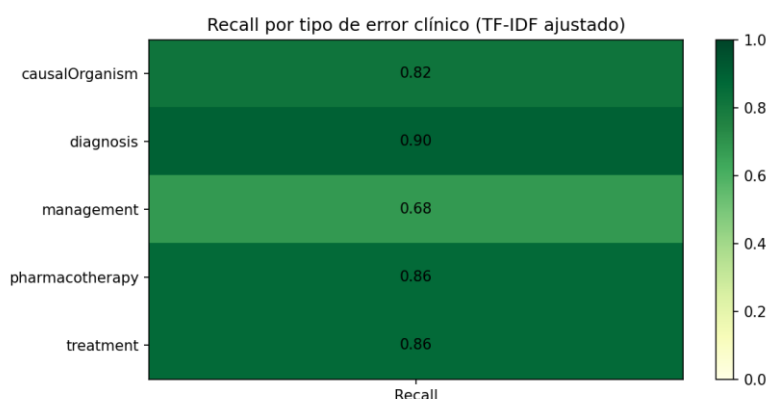


Figura 3. Recall por ErrorType (TF-IDF).

3.4. Verificación FastAPI y calidad del código (S10)

Verificación operativa del backend (23-ago-2026):

Con ``uvicorn api.main:app --port 8000``, GET `/health` respondió `status=ok` y `modelo_tfidf_disponible=true` (`salidas_ajuste/modelo_ajustado.joblib`). POST `/generar` con nota de alergia a penicilina y prescripción de amoxicilina (`mock_llm=true`) devolvió HTTP 200; `top1="Se indica amoxicilina..."`, `alerta=true`, brazos `tfidf + llm_zero + llm_rag` activos.

Documentación interactiva en `/docs`. Frontend React (`frontend/`, puerto 5173) consume la misma API vía proxy Vite.

Refactor del pipeline CITIMED (code review S10): `s10/citimed_utils.py` centraliza extracción del cuerpo clínico, detección heurística de PHI residual y lectura de salidas anonimizadas (`.txt/.pdf`). `s10/probar_citimed.py` separa eval cross-domain (solo plantilla etiquetada) de inferencias_piloto; aborta si queda PHI (`--force` solo depuración). `demo/ejemplos.py` reutiliza `citimed_utils`.

Tests automatizados:

`s10/test_citimed_pipeline.py` (6 casos pytest: PHI, CSV, cabeceras).

Controles de repositorio (`.gitignore`): `data/citimed_odontologia.csv` generado, `s10/anonizador/salidas/`, `historia_ejemplo.txt` con PHI aparente (uso local); en git solo ejemplos sintéticos (`ejemplos/historia_ejemplo sintetico.txt`) y salidas anonimizadas de demostración. Entrega S10: consolidación en rama main pendiente de commit/push final con anonimizador, cleanup CITIMED e informe regenerado.

Comparación tripartita (test MEDEC, s7/eval_tripartita.py):

- TF-IDF: ROC-AUC=0,954, AUPRC=0,475, latencia≈0.41 ms/oración.
- LLM zero-shot (mock): ROC-AUC=0,498 (sin discriminación en mock; evaluación con API/Ollama pendiente).
- LLM+RAG: ROC-AUC=0,498.

Análisis por ErrorType (TF-IDF): causalOrganism: recall=0,818, loc=0,818; diagnosis: recall=0,897, loc=0,948; management: recall=0,680, loc=0,732; pharmacotherapy: recall=0,861, loc=0,861; treatment: recall=0,863, loc=0,824.

Sesgo EN-ES (subset n=200): Δ AUC=0,000. TF-IDF stop_words español: test AUC=0,959.

4. Discusión

Los resultados sostienen con evidencia empírica la arquitectura propuesta. El fracaso del baseline no es un revés sino una demostración: las inconsistencias clínicas no son patrones léxicos, sino contradicciones semánticas frente al conocimiento médico, y por tanto requieren un modelo capaz de razonar sobre el contenido y de contrastarlo con fuentes externas —justamente lo que ofrece un enfoque LLM + RAG—. El modelo ajustado confirma que, cuando la unidad de análisis es la adecuada (la oración), incluso un método clásico recupera señal y localiza el error con alta fiabilidad, lo que valida el pipeline de datos y evaluación que reutilizará la solución final.

En términos de utilidad organizacional, la capacidad de localización es la más valiosa para CITIMED: el asistente no debe limitarse a decir “esta nota tiene un error”, sino “revise esta frase”, reduciendo la carga del médico. La baja precisión a nivel de oración, consecuencia del fuerte desbalance, se gestionará ajustando el umbral de decisión según la carga de revisión aceptable. La principal limitación actual es el uso de un corpus en inglés; la transferencia al español y al contexto de CITIMED es la prioridad de la siguiente fase.

5. Interpretabilidad (preliminar)

La interpretabilidad es un requisito, no un adorno: un asistente clínico debe justificar cada alerta. El proyecto la aborda en tres niveles. Primero, coeficientes TF-IDF auditable en Regresión Logística. Segundo, localización top-1 del 84,6 % (oración concreta verificable). Tercero, explicaciones SHAP/LIME (s7/interpretabilidad.py) y contexto RAG citado en la demo y la API.

En LLM+RAG, la demo muestra fragmentos recuperados de guías clínicas (s7/knowledge/) junto a cada score. Faithfulness RAG formal y corrección sugerida automática quedan como trabajo futuro; las figuras SHAP/LIME están en s10/evidencias/capturas/.

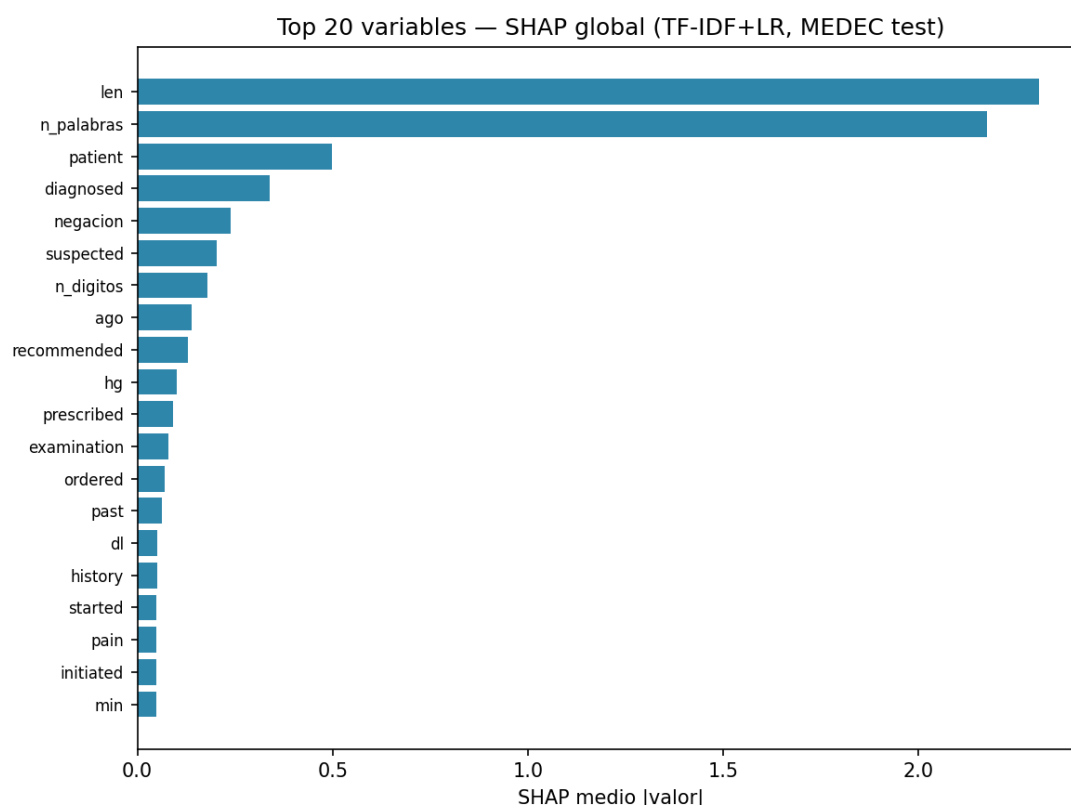


Figura 4. SHAP summary (TF-IDF).

Interpretabilidad entregada (S7-S10):

- SHAP/LIME sobre TF-IDF: s7/interpretabilidad.py → salidas_s7/shap_*.png, lime_*.png (ver s7/docs/reporte_interpretabilidad_etica.md).
- Demo Streamlit y API FastAPI muestran: oración top-1, scores por brazo, contexto RAG recuperado (demo/components/results_panel.py).
- Formato de reporte al médico: oración señalada + tipo de error + scores + evidencia RAG; corrección sugerida automática: plan futuro.

6. Riesgos éticos (preliminar)

- **Privacidad y protección de datos.** Las historias clínicas son datos sensibles. Mitigación: anonimización previa, cumplimiento de la Ley Orgánica de Protección de Datos Personales del Ecuador, aval del comité de ética/bioética, cifrado en reposo y procesamiento local (los datos del paciente no salen de la institución ni se envían a servicios externos).
- **Sesgo y equidad.** Los LLM pueden reproducir sesgos demográficos, y la evidencia muestra que también existen en modelos en español. Mitigación: auditar el desempeño por subgrupos y evaluar el sesgo en el idioma de las historias.
- **Alucinación y confianza excesiva.** El modelo podría afirmar errores inexistentes o pasar por alto reales. Mitigación: anclaje en fuentes (RAG), supervisión humana obligatoria (human-in-the-loop) y un umbral prudente que priorice el recall en errores críticos.
- **Rol de apoyo, no de decisión.** La responsabilidad clínica permanece en el profesional; el sistema asiste y documenta, no decide.

Riesgos y mitigaciones implementadas:

- Servicio de anonimización local (s10/anonizador/): reglas + NER, pseudonimización consistente, fechas desplazadas, auditoría con hashes (sin PHI en claro). Revisión humana por muestreo recomendada antes de liberar datos.

- Mock LLM por defecto; consentimiento PHI en demo (demo/components/security.py).
- .gitignore: CSV/salidas CITIMED generadas, historia_ejemplo.txt local; solo ejemplos sintéticos versionados (ejemplos/historia_ejemplo.sintetico.txt).
- Pipeline aborta si PHI residual (s10/citimed_utils.py); tests en s10/test_citimed_pipeline.py.
- Auditoría SHA-256 sin texto clínico (salidas_s7/audit.log).
- Fallback TF-IDF si falla API LLM (s7/test_fallback.py; modo_degradado=True).
- Evaluación sesgo EN-ES: s7/eval_idioma.py, s7/eval_tfidf_idioma.py.
- Protocolo formal CITIMED + aprobación comité de ética: pendiente organizacional.

7. Conclusiones y trabajo restante

7.1. Respuesta a observaciones de la Semana 7

En la retroalimentación de la Semana 7 se identificaron dos deudas técnicas centrales para el despliegue en CITIMED: (i) la brecha inglés–español del corpus y del pipeline, y (ii) el desbalance extremo de la tarea a nivel de oración (prevalencia $\approx 4,5\%$ de errores en MEDEC), que limita la utilidad de métricas como accuracy y exige priorizar AUPRC y recall por tipo de error.

Durante S10 se avanzó en el diagnóstico y en la infraestructura para abordar esas deudas, sin cerrar aún la validación clínica en español a escala. Concretamente: (a) se implementaron evaluaciones de sesgo EN–ES (s7/eval_idioma.py, s7/eval_tfidf_idioma.py), confirmando que stop words en español mejoran el TF-IDF respecto a configuraciones solo en inglés; (b) se reporta AUPRC junto a ROC-AUC y se usa `class_weight="balanced"` en la regresión logística; (c) se inició el camino hacia corpus español con el piloto CITIMED (anonimización local, pipeline s10/probar_citimed.py, inferencia en español y eval cross-domain sobre plantilla odontológica etiquetada). Estas acciones mitigan el riesgo, pero no sustituyen un corpus CITIMED anotado de cientos o miles de oraciones.

A continuación se alinea la recomendación docente de S7 con el estado del proyecto y el plan inmediato post-S10:

Recomendación S7	Avance hasta S10	Próximo paso (priorizado)
(1) Aumentar datos en español vía CITIMED anonimizado (meta: 500–1000 oraciones, etiquetado parcial)	Anonimizador operativo; pipeline reproducible; piloto con historia sintética anonimizada e inferencia segmentada; eval sobre plantilla odontológica (n pequeño)	Lote institucional anonimizado + etiquetado parcial de inconsistencias (medicación, diagnóstico/plan); ampliar CSV CITIMED y re-evaluar cross-domain y fine-tuning
(2) Features específicas para odontología (p. ej. BioWordVec, SapBERT) en lugar de solo TF-IDF	TF-IDF + rasgos numéricos siguen siendo el baseline fuerte (ROC-AUC $\approx 0,949$ en MEDEC); RAG usa embeddings solo para recuperación, no como clasificador	Piloto comparativo: SapBERT / BioClinicalBERT vs TF-IDF en subset odontológico CITIMED; métrica principal AUPRC y recall en farmacoterapia

(3) Manejo explícito del desbalance (SMOTE, weighted loss) además de umbrales	class_weight="balanced"; AUPRC reportada; umbral configurable en demo/API	Experimento con SMOTE a nivel de oración (train MEDEC y/o CITIMED) vs balanced LR; calibración de umbral por prevalencia estimada en CITIMED
--	---	--

En síntesis, S10 responde parcialmente a S7: se cuantificó el sesgo idioma, se priorizó AUPRC frente al desbalance y se habilitó la ruta de datos en español. La principal deuda técnica persiste hasta contar con corpus CITIMED anotado (500–1000 oraciones como mínimo razonable) y con experimentos de embeddings clínicos y rebalanceo más allá del ajuste de umbrales. Esas tres líneas constituyen el eje del trabajo inmediato posterior a esta entrega

7.2. Hasta la semana 10, el proyecto cuenta con un problema bien delimitado, arquitectura implementada y evaluada sobre MEDEC, pipeline reproducible (s6/, s7/, api/, demo/) y línea base cuantitativa: ROC-AUC 0,949, AUPRC 0,419, localización 84,6 %. La comparación tripartita confirma ventaja de TF-IDF en modo mock (LLM AUC≈0,5); evaluación con API/Ollama real pendiente. Como avance CITIMED (no corpus final), se validó anonimización local, inferencia piloto sobre 1 historia(s) anonimizada(s) (10 oraciones, sin mezclar en métricas AUC) y eval cross-domain sobre plantilla odontológica etiquetada (n=4): ROC-AUC 1,000, AUPRC 1,000. API FastAPI verificada (GET /health, POST /generar con alerta en ejemplo medicación).

El criterio de éxito para piloto clínico: corpus anotado con aval ético, mantener recall en farmacoterapia/diagnóstico, desplegar con Ollama on-premise y fallback transparente.

Trabajo futuro (post-S10): corpus CITIMED clínico anotado con aval ético, ampliar lote anonimizado, eval tripartita con API/Ollama real, cascada TF-IDF→LLM codificada, Docker/DVC/MLflow, corrección sugerida y faithfulness RAG.

#	Tarea	Estado	Sprint
1	Comparación tripartita: baseline vs. LLM zero-shot vs. LLM + RAG	Implementado (S7/S10)	S10
2	Índice FAISS de guías, CIE-10 y vademécum; servicio de recuperación	Implementado (s7/rag_index.py)	S10
3	MLflow operativo y pipeline de reentrenamiento documentado	Plan futuro	S10
4	Corpus de CITIMED anonimizado y anotado en español	Piloto S10 (anonimizador + 1 historia)	S10
5	Interpretabilidad con cita de fuente y faithfulness; auditoría de sesgo	Implementado (SHAP/LIME + RAG demo)	S10
6	Redacción final, portada, índice y revisión de coherencia	S10 consolidado	S10

Referencias

- Amugongo, L. M., Mascheroni, P., Brooks, S., Doering, S., & Seidel, J. (2025). Retrieval augmented generation for large language models in healthcare: A systematic review. *PLOS Digital Health*, 4(6), e0000877.
- Ben Abacha, A., Yim, W., Fu, Y., Sun, Z., Yetisgen, M., Xia, F., & Lin, T. (2025). MEDEC: A benchmark for medical error detection and correction in clinical notes. *Findings of the ACL 2025*.
- Cai, Q., Yang, L., Xiao, J., Ma, J., Liu, M., & Pan, X. (2025). Train-time and test-time computation in LLMs for error detection and correction in electronic medical records. *Diagnostics*, 15(14), 1829.
- Shah, S. V. (2024). Accuracy, consistency, and hallucination of large language models when analyzing unstructured clinical notes in electronic medical records. *JAMA Network Open*, 7(8), e2425953.
- Xiong, G., Jin, Q., Lu, Z., & Zhang, A. (2024). Benchmarking retrieval-augmented generation for medicine. *Findings of the ACL 2024*.
- Zakka, C., Shad, R., Chaurasia, A., Dalal, A. R., Kim, J. L., Moor, M., et al. (2024). Almanac — Retrieval-augmented language models for clinical medicine. *NEJM AI*, 1(2).

Anexo — Evidencias técnicas (repositorio)

Todos los resultados son reproducibles desde la raíz del repositorio MIA_Deteccion_HC/ (SEED=42, particiones oficiales MEDEC).

Estructura: s6/ (TF-IDF), s7/ (LLM, RAG, evals), api/ (FastAPI), frontend/ (React), demo/ (Streamlit), s10/ (evidencias, informe, anonimizador). Salidas: salidas_ajuste/, salidas_s7/ (incl. prueba_citimed.json). Comandos: python s6/modelo_ajustado.py; python s7/eval_tripartita.py -- mock-llm --max-oraciones 500; python s7/test_fallback.py; python s10/probar_citimed.py; python -m pytest s10/test_citimed_pipeline.py -q; streamlit run demo/app.py; uvicorn api.main:app --port 8000 (verificar GET /health y POST /generar en /docs).

Evidencias: s10/evidencias/. Repositorio: https://github.com/marcoz1691/MIA_Deteccion_HC.
Nota entrega: commit final S10 (anonimizador + cleanup CITIMED + informe) pendiente en git.

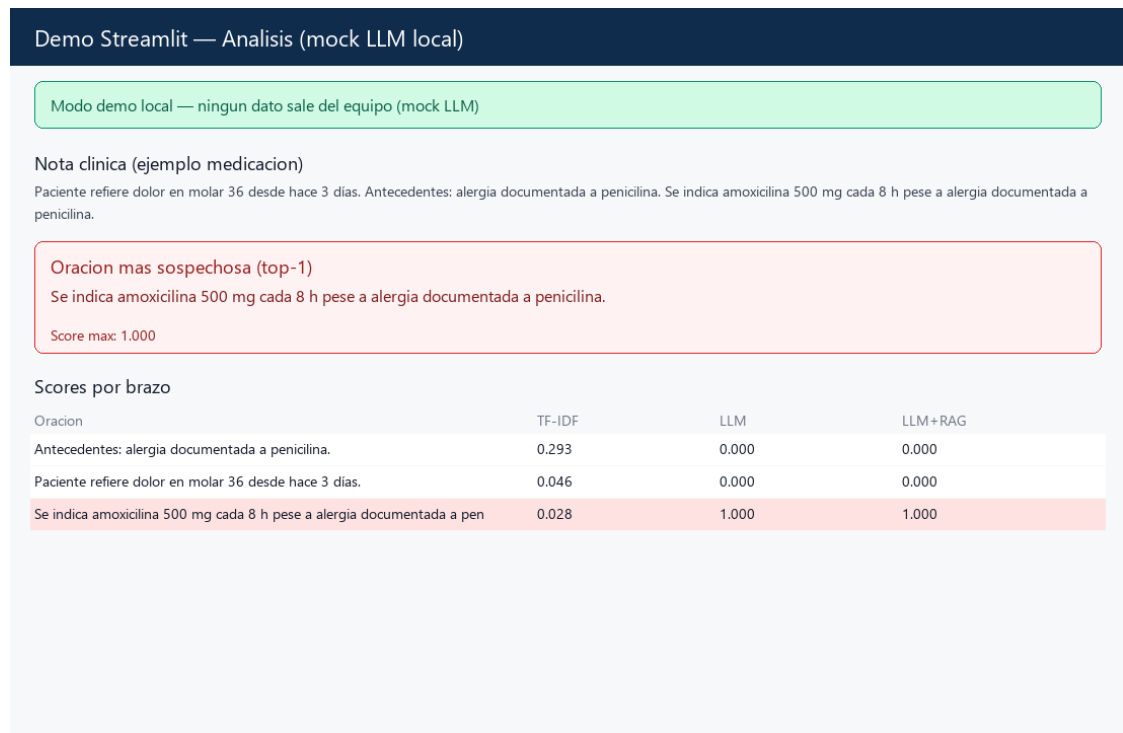


Figura 5. Demo — análisis ejemplo medicación.

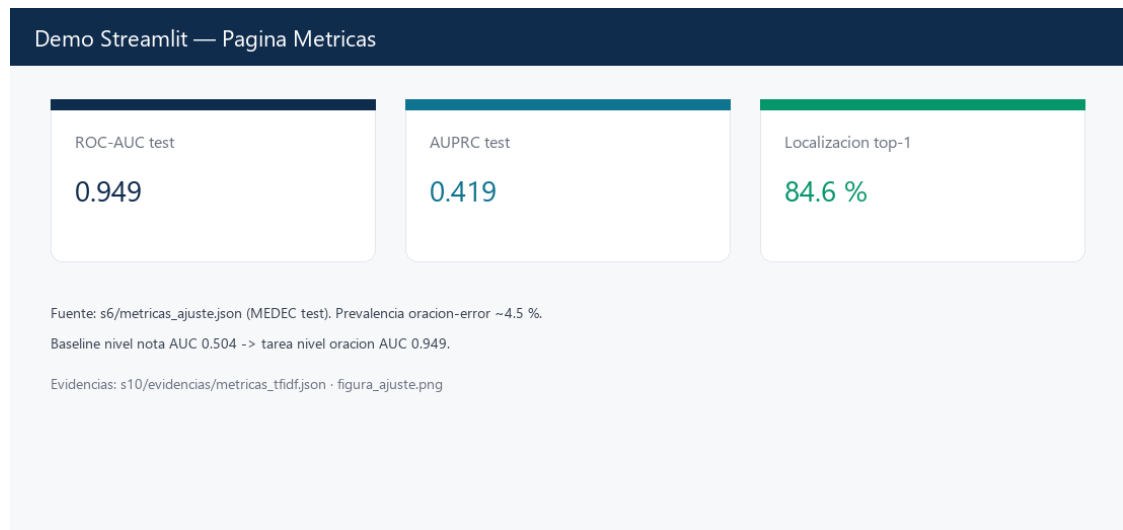


Figura 6. Demo — métricas ROC-AUC / AUPRC.

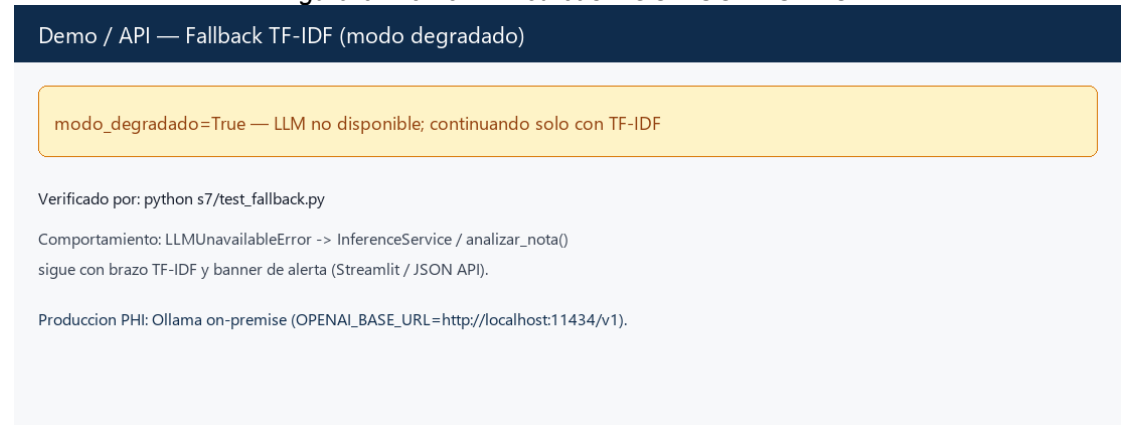


Figura 7. Modo degradado / fallback TF-IDF.

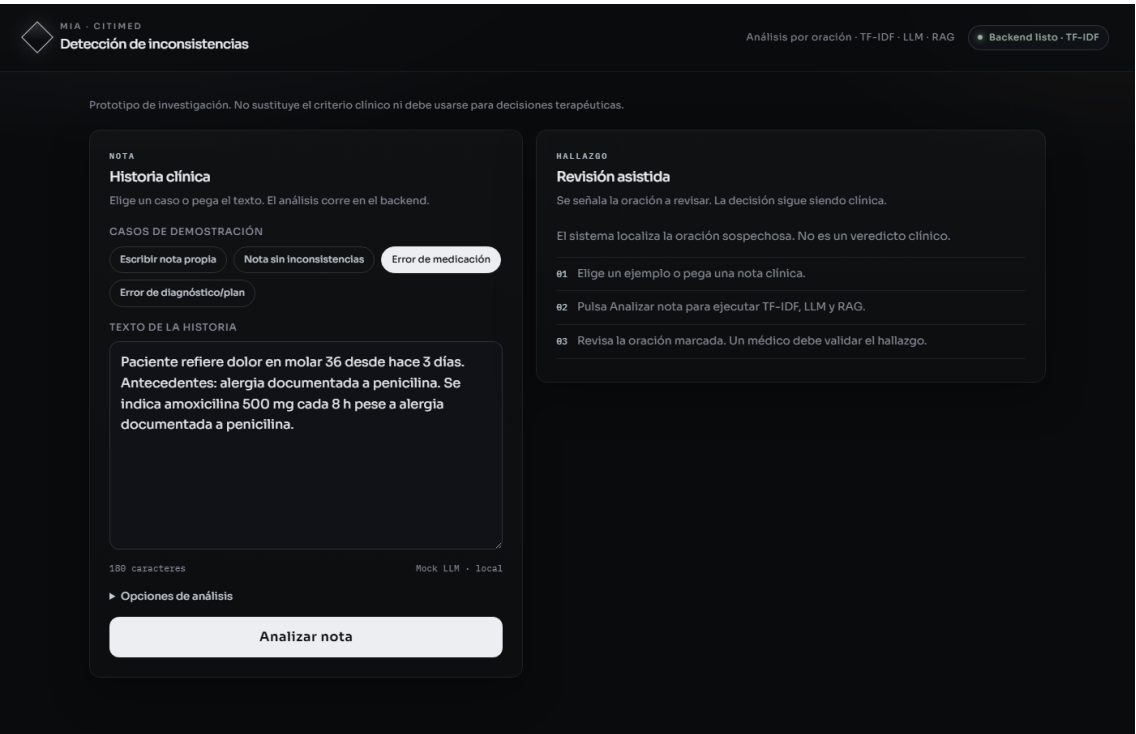


Figura 8. Front End levantado con fastApi – prueba de ingresar historia clínica.

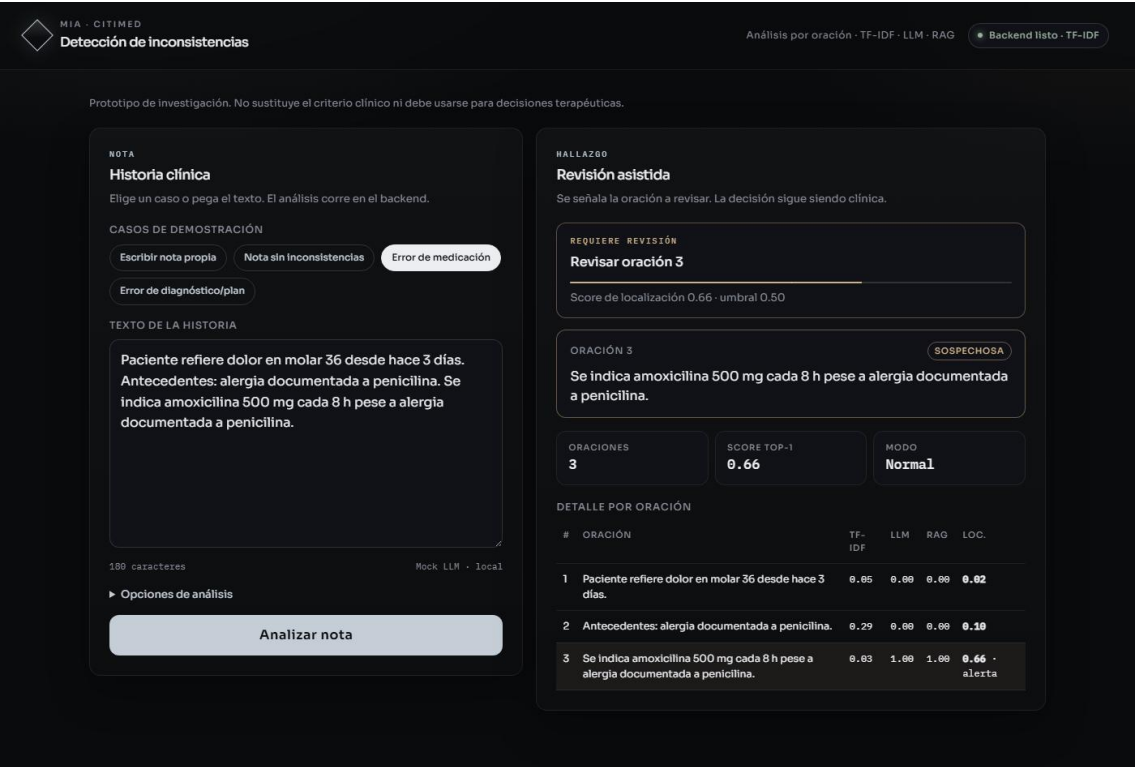


Figura 9. Front End levantado con fastApi – prueba de análisis de historia clínica.